SOC Home Lab: Open-Source SIEM Deployment & Threat Detection

1. Summary

As part of my cybersecurity learning journey, I aimed to move beyond theoretical knowledge and gain hands-on experience in security monitoring, log analysis, and incident detection. While studying Security Operations Center (SOC) concepts, SIEM technologies, and defensive security practices, I realized that the most effective way to understand these topics was to build and operate my own homelab environment.

To achieve this, I designed and deployed a small SOC homelab consisting of three virtual machines. The environment includes an Ubuntu server running the Wazuh platform as the central SIEM solution, a Windows 10 workstation used as the primary endpoint, and a Kali Linux machine used to simulate attacker activity. In addition, the Ubuntu Wazuh server is extended with a network intrusion detection system (IDS) to provide network-level visibility alongside endpoint monitoring. To further extend the lab with web application attack capabilities, DVWA (Damn Vulnerable Web Application) was deployed on the Windows 10 workstation using XAMPP, providing a realistic and intentionally vulnerable web application target for simulating attacks such as SQL injection, command injection, and cross-site scripting. All systems are connected through an isolated virtual network to ensure safe and controlled testing.

The main objective of this project was to understand how security events are generated, collected, analyzed, and detected in a realistic but simplified enterprise-like environment. Instead of focusing only on tool installation, I aimed to understand the full detection lifecycle, including log generation, log forwarding, alert creation, and incident analysis.

Throughout the project, I focused on three key areas. The first was visibility, ensuring that the Windows 10 system generated meaningful telemetry through Windows Event Logs and Sysmon, while the Ubuntu server provided centralized log collection and network-based detection through IDS integration. Apache access logs generated by DVWA were also forwarded to the Wazuh SIEM, extending visibility to the web application layer. The second was attack simulation, where I used Kali Linux to perform controlled activities such as network reconnaissance, brute-force attempts, basic exploitation techniques, and web application attacks against DVWA. The third was detection and analysis, where I observed how Wazuh and the IDS system processed events, generated alerts, and correlated suspicious behavior across different data sources.

This project allowed me to gain practical experience with SIEM operations, endpoint monitoring, network intrusion detection, web application security, threat simulation, and log analysis. It also helped me understand how security analysts investigate alerts and differentiate between normal and malicious activity in a monitored environment.

The following report presents the architecture of the lab, the technologies used, the configuration process, attack simulations performed, and the security events and alerts observed throughout the project. The goal of this work is to demonstrate a practical understanding of SOC operations and defensive security concepts through hands-on implementation and analysis.

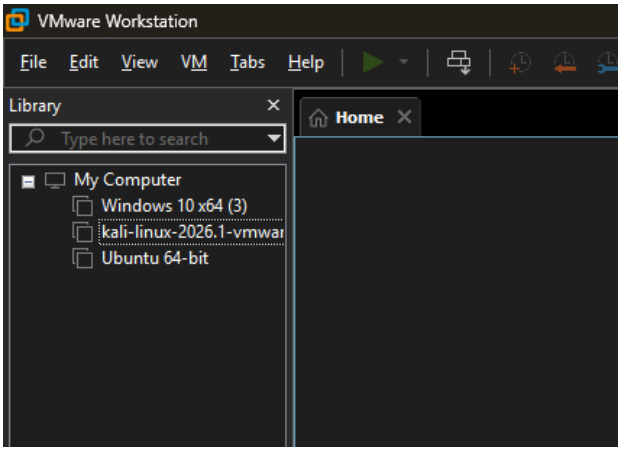
2. Architecture Overview

2.1 Why I Use 3 Virtual Machines

I designed this SOC homelab to simulate a realistic but lightweight security monitoring environment that could run on a personal machine. Three virtual machines provided the right balance between functionality, performance, and

simplicity. The environment includes a centralized monitoring and detection server, a Windows-based endpoint, and an attacker machine used for generating security events.

The goal was not to build a large or complex enterprise network, but to clearly understand how security telemetry is generated, transmitted, and analyzed across endpoint and network layers. By keeping the architecture minimal, I was able to focus more on detection logic, log analysis, and attack interpretation rather than infrastructure overhead.



2.2 Roles of Each Machine

- **Wazuh Server (Ubuntu Server + IDS)**

This machine runs the Wazuh Manager, Indexer, and Dashboard, acting as the central Security Information and Event Management (SIEM) platform for the lab. It collects logs from endpoints, processes security events, applies detection rules, and generates alerts for suspicious activity.

In addition to Wazuh, this server also runs a Network Intrusion Detection System (IDS), providing visibility into network traffic and detecting malicious patterns such as port scanning, suspicious payloads, and exploit attempts. This combination allows correlation between endpoint events and network-based detections, improving overall visibility of attacker behavior.

- **Windows 10 Workstation**

This machine represents a typical user endpoint in a corporate environment. It is joined to the lab network and generates normal user activity such as logins,

process execution, PowerShell usage, and network connections. The Wazuh agent and Sysmon are installed to provide detailed endpoint telemetry, which is forwarded to the Wazuh server for centralized analysis and detection. In addition, DVWA was deployed on this machine using XAMPP as a local web server, turning the endpoint into a vulnerable web application target. This allowed realistic web attack simulations to be performed from Kali Linux while all HTTP traffic was monitored through Apache access logs forwarded to the Wazuh SIEM.

- **Kali Linux Machine**

This machine is used to simulate attacker behavior and generate controlled security events within the environment. It performs activities such as network reconnaissance, scanning, brute-force attempts, and basic exploitation techniques against the Windows endpoint. The purpose of this machine is to produce realistic attack traffic that can be detected, analyzed, and correlated within the SIEM and IDS systems.

2.3 Network Flow Overview

All machines in the lab are connected using a combination of Host-Only networking and NAT to simulate a controlled and isolated SOC environment while still allowing limited external connectivity when needed for updates and tool installation.

The Host-Only network is used as the primary communication channel between the virtual machines. All internal traffic, including attack simulation and log generation, flows through this isolated network. This ensures that all security events remain contained within the lab environment and can be safely monitored and analyzed without exposing the host system.

The NAT network is used on selected machines to allow internet access for package installation, updates, and tool downloads. However, it is not used for internal attack traffic or security monitoring. This separation helps maintain a realistic lab structure where internal enterprise traffic is isolated from external internet communication

3. Build Steps

This section describes how I built the SOC homelab, from creating the virtual machines to validating security events and alerts within the Wazuh dashboard. The objective was to create a practical environment for monitoring endpoint and network activity while simulating real-world attack scenarios.

Step 1: Create the Virtual Machines

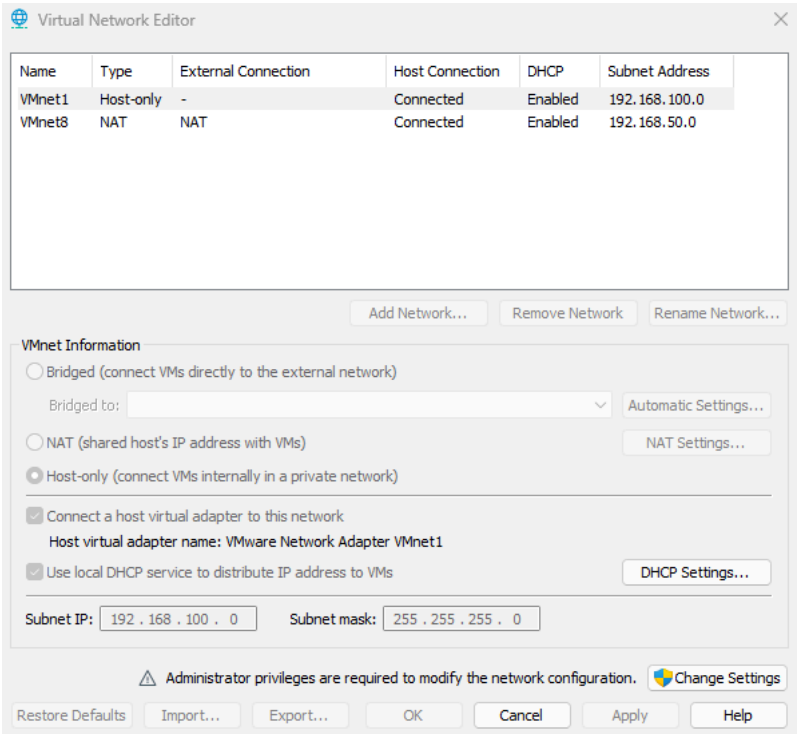
I used VMware to create three virtual machines: an Ubuntu Server running Wazuh and Suricata, a Windows 10 workstation acting as the monitored endpoint, and a Kali Linux machine used for attack simulation. Each virtual machine was allocated sufficient CPU, memory, and storage resources to ensure stable operation while running security tools and monitoring services.

The Ubuntu server hosts both the Wazuh platform and the network intrusion detection system (IDS). The Windows 10 machine serves as the target endpoint, while Kali Linux is used to generate attack traffic and security events.

Step 2: Configure the Network

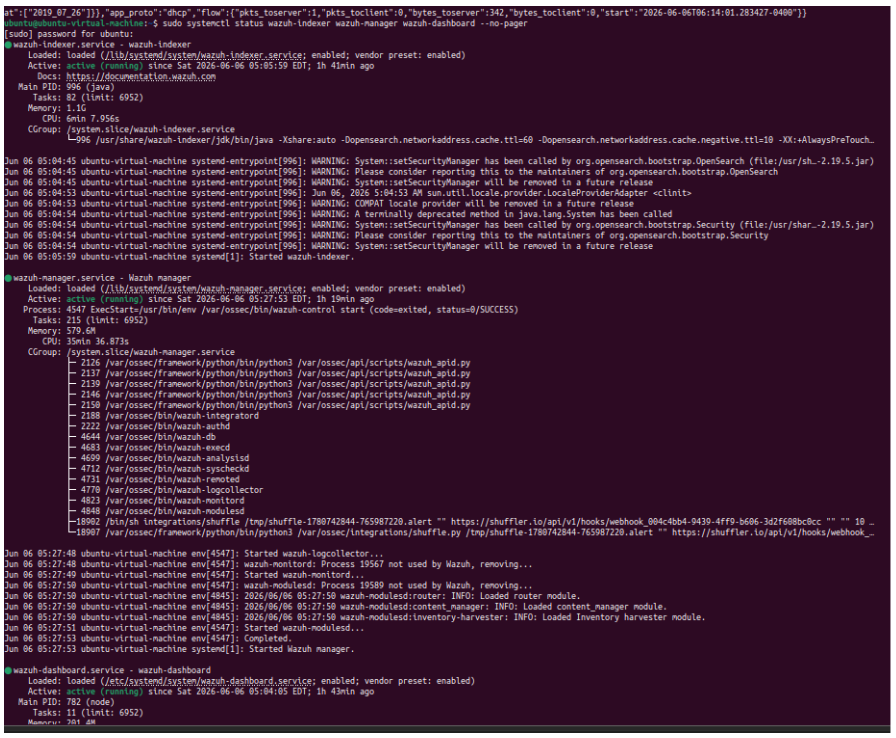
The lab environment uses a combination of Host-Only and NAT networking.The Host-Only adapter provides an isolated network where all attack simulations, monitoring activities, and communication between virtual machines take place. This ensures that all traffic remains within the lab environment.

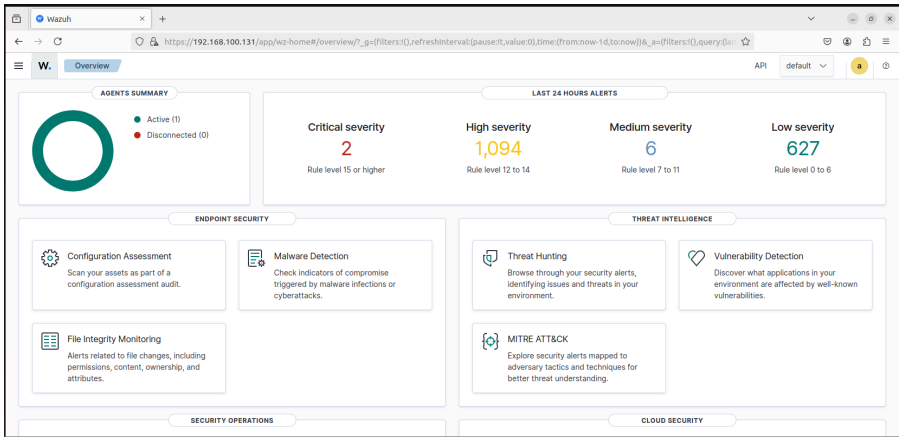
The NAT adapter provides internet access for downloading updates, installing packages, and obtaining security tools when necessary. Internal attack traffic does not traverse the NAT network.



Step 3: Install and Configure Wazuh

On the Ubuntu server, I installed the Wazuh Manager, Indexer, and Dashboard using the official installation method. After installation, I verified that all Wazuh services were running correctly and confirmed that the dashboard was accessible through a web browser.I also verified that the Wazuh Manager was listening for agent connections and successfully receiving security events from monitored systems.





Step 4: Install Sysmon and the Wazuh Agent on Windows 10

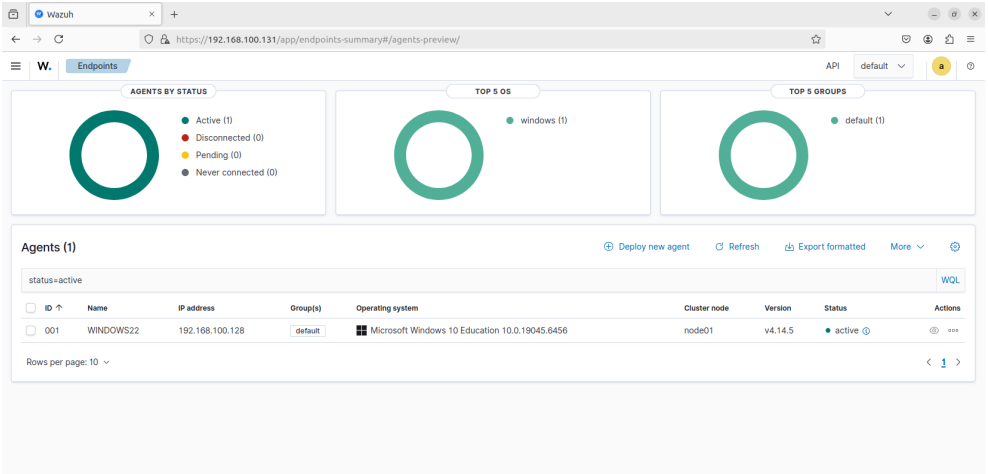
On the Windows 10 workstation, I installed Sysmon to generate detailed endpoint telemetry and improve visibility into system activity.

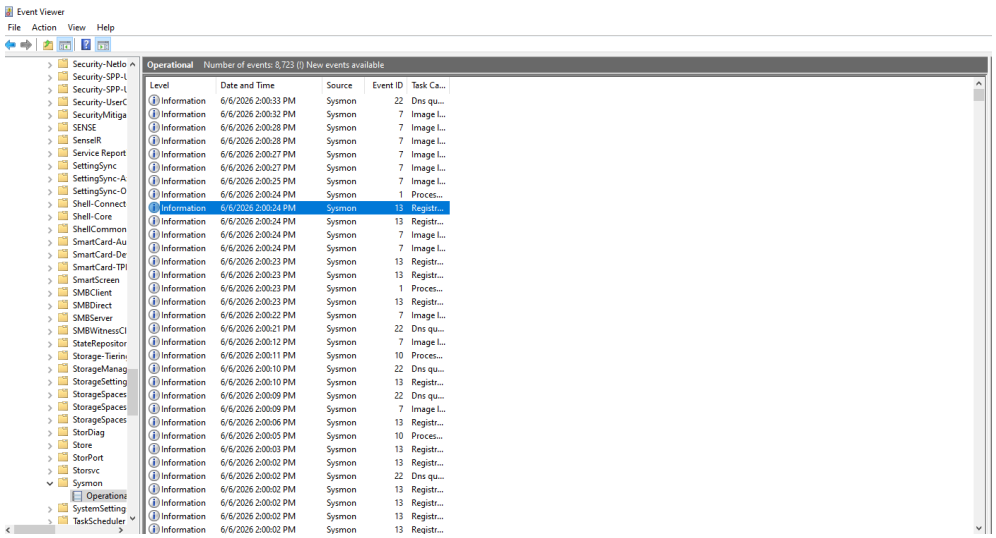
Sysmon was configured to capture events such as:

- Process creation

- Network connections
- PowerShell execution
- File creation and modification
- Registry activity

After Sysmon installation, I deployed the Wazuh agent and enrolled it with the Wazuh Manager. Once connected, the agent began forwarding Windows Event Logs and Sysmon events to the SIEM platform for analysis.



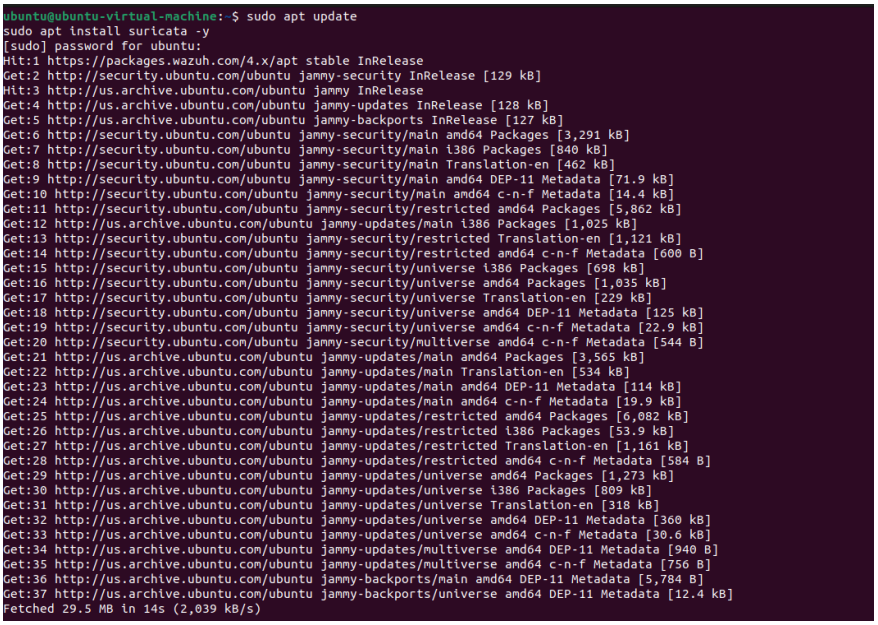


Step 5: Install and Configure Suricata IDS

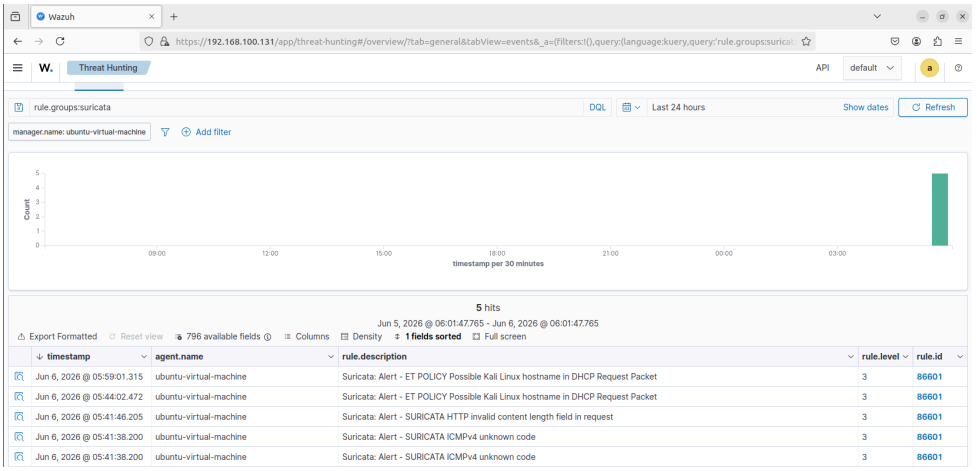
To add network-level visibility to the environment, I installed Suricata on the Ubuntu server. Suricata was configured to monitor traffic flowing through the Host-Only network and generate alerts for suspicious activity.

I verified the IDS functionality by generating test traffic and confirming that Suricata produced network alerts and stored them in its logging system. These alerts were later integrated into the overall monitoring workflow.

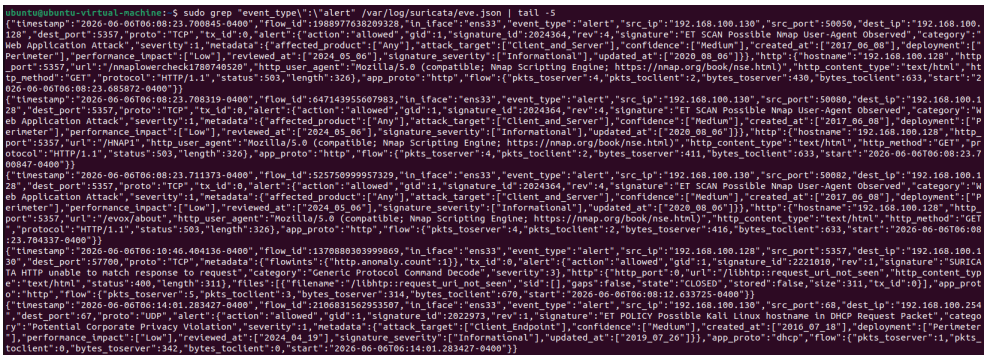
- We installed Suricata on the Ubuntu Wazuh server and configured it by setting the monitored interface to ens33, defining HOME_NET as 192.168.100.0/24, fixing the rules path to /var/lib/suricata/rules/suricata.rules, and adding the eve.json log file to Wazuh's ossec.conf to enable alert ingestion.

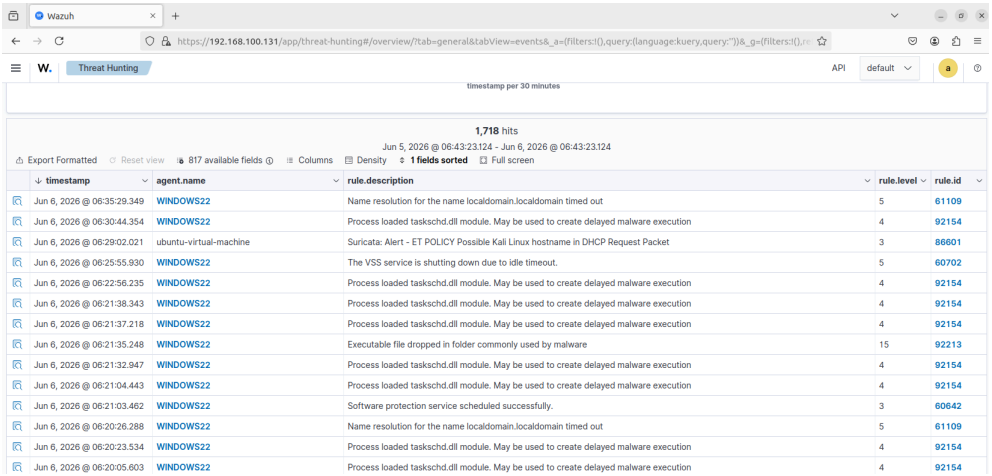


- We ran an Nmap scan from Kali against the Ubuntu Wazuh server (`192.168.100.131`) and confirmed the alerts appeared in the Wazuh Dashboard under Threat Hunting filtered by `rule.groups:suricata`.



- We ran an Nmap scan from Kali targeting the Windows machine (`192.168.100.128`) and confirmed Suricata detected it





timestamp	agent.name	rule.description	rule.level	rule.id
Jun 6, 2026 @ 06:35:29.349	WINDOWS22	Name resolution for the name localdomain.localdomain timed out	5	61109
Jun 6, 2026 @ 06:30:44.354	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:29:02.021	ubuntu-virtual-machine	Suricata: Alert - ET POLICY Possible Kali Linux hostname in DHCP Request Packet	3	86601
Jun 6, 2026 @ 06:25:55.930	WINDOWS22	The VSS service is shutting down due to idle timeout.	5	60702
Jun 6, 2026 @ 06:22:56.235	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:21:38.343	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:21:37.218	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:21:35.248	WINDOWS22	Executable file dropped in folder commonly used by malware	15	92213
Jun 6, 2026 @ 06:21:32.947	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:21:04.443	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:21:03.462	WINDOWS22	Software protection service scheduled successfully.	3	60642
Jun 6, 2026 @ 06:20:26.288	WINDOWS22	Name resolution for the name localdomain.localdomain timed out	5	61109
Jun 6, 2026 @ 06:20:23.534	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Jun 6, 2026 @ 06:20:05.603	WINDOWS22	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154

Step 6: Deploy DVWA on Windows 10

To extend the lab with web application attack simulation capabilities, DVWA (Damn Vulnerable Web Application) was deployed on the Windows 10 workstation using XAMPP as the local web server stack.

XAMPP was installed on the Windows 10 machine, providing Apache and MySQL services. After confirming that Apache and MySQL were running correctly through the XAMPP Control Panel, DVWA was downloaded from its official GitHub repository and placed in the XAMPP web root directory at C:\xampp\htdocs\dvwa.

DVWA was then configured by editing config.inc.php to point to the local MySQL instance, and the DVWA database was created through phpMyAdmin. The setup page at http://localhost/dvwa/setup.php was used to initialize the database, and DVWA was confirmed accessible via the login page using the default credentials.

To allow attack simulation from the Kali Linux machine, Apache was configured to accept connections from the Host-Only network by modifying httpd.conf to allow all granted access. Access from Kali was verified by browsing to http://192.168.100.128/dvwa from the Kali machine.

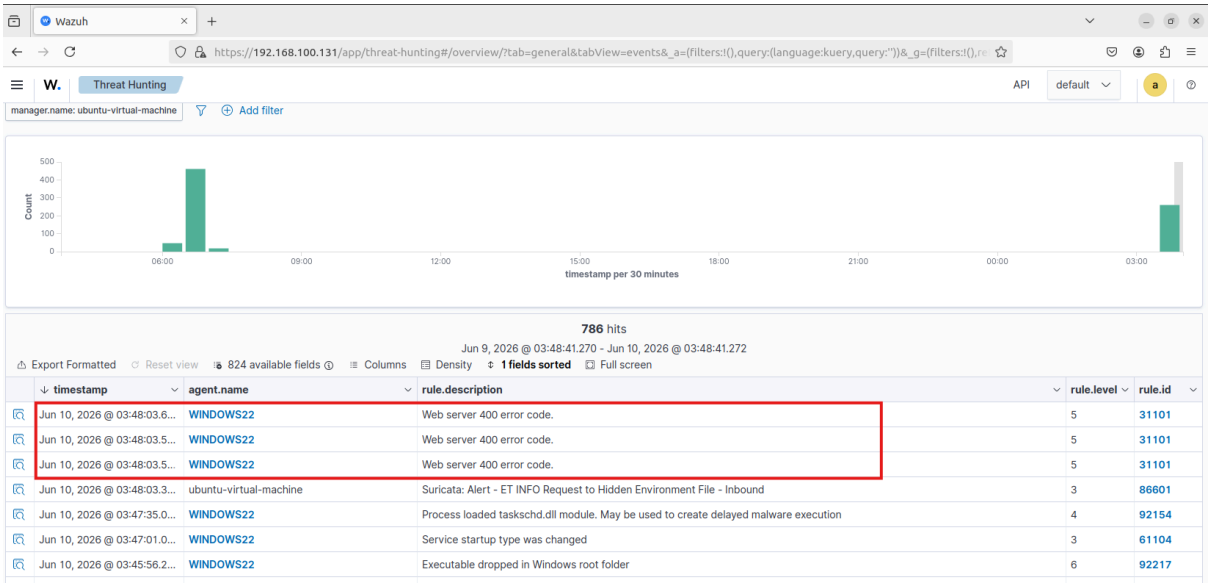
Finally, Wazuh agent log monitoring was extended to include Apache access and error logs by adding the following configuration to ossec.conf on the Windows 10 machine:

```
ossec - Notepad
File  Edit  Format  View  Help
</localfile>

<localfile>
  <location>active-response\active-responses.log</location>
  <log_format>syslog</log_format>
</localfile>

<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>C:\\xampp\\apache\\logs\\access.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>C:\\xampp\\apache\\logs\\error.log</location>
</localfile>

<!-- Policy monitoring -->
<rootcheck>
  <disabled>no</disabled>
  <windows_apps>./shared/win_applications_rcl.txt</windows_apps>
  <windows_malware>./shared/win_malware_rcl.txt</windows_malware>
</rootcheck>

<!-- Security Configuration Assessment -->
<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
```

Test HTTP request was sent from Kali Linux targeting the DVWA application, simulating an attacker probing the web server:

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.100.128/dvwa/nonexistentpage.php
curl http://192.168.100.128/admin
curl http://192.168.100.128/.env
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at 192.168.100
.128 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at 192.168.100
.128 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at 192.168.100
.128 Port 80</address>
</body></html>
```

The request was logged by Apache, forwarded by the Wazuh agent to the Wazuh Manager, and confirmed visible in the Wazuh Dashboard under agent WINDOWS22:



Step 7: Prepare Kali Linux for Attack Simulation

On the Kali Linux machine, I verified the availability of common penetration testing and assessment tools used for generating security events within the lab.

Step 8: Validate Log Collection and Alert Generation

Once all components were deployed, I performed several test activities from Kali Linux against the Windows 10 workstation to validate the monitoring pipeline.

I verified that:

- Windows Event Logs were collected successfully
- Sysmon events appeared within the Wazuh dashboard
- Suricata generated network-based alerts
- Wazuh processed and displayed security events correctly
- Endpoint and network activity could be correlated during investigations
- Apache access logs generated by DVWA were forwarded by the Wazuh agent and appeared in the Wazuh Dashboard under agent WINDOWS22

After confirming proper functionality, I reviewed the generated alerts, adjusted configurations where necessary, and ensured that the monitoring environment provided clear visibility into both normal and suspicious activity.

4. Attack Scenarios

4.0 Introduction

This section documents four controlled attack simulations performed from the Kali Linux machine against the Windows 10 endpoint. Each scenario was designed to generate realistic security events that could be detected and analyzed within the Wazuh SIEM and Suricata IDS. The scenarios cover network reconnaissance, credential-based attacks, and web application attacks including SQL injection, cross-site scripting.

4.1 Attack Scenario — Reconnaissance Via Nmap

Purpose

The purpose of this scenario was to simulate an attacker performing initial network reconnaissance against the Windows 10 endpoint and verify whether Suricata IDS could detect port scanning activity. Network reconnaissance is almost always the first step in any real attack; an attacker needs to know what services are running before attempting exploitation. If the IDS cannot detect a basic Nmap scan, it provides no early warning capability whatsoever.

Setup

Field	Detail
Target Machine	Windows 10 Workstation

	(192.168.100.128)
Attacker Machine	Kali Linux (192.168.100.130)
Monitoring	Suricata IDS on Ubuntu Server + Wazuh Manager
Service Attacked	All open ports — full TCP SYN scan

Rule Triggered (Custom Rule)

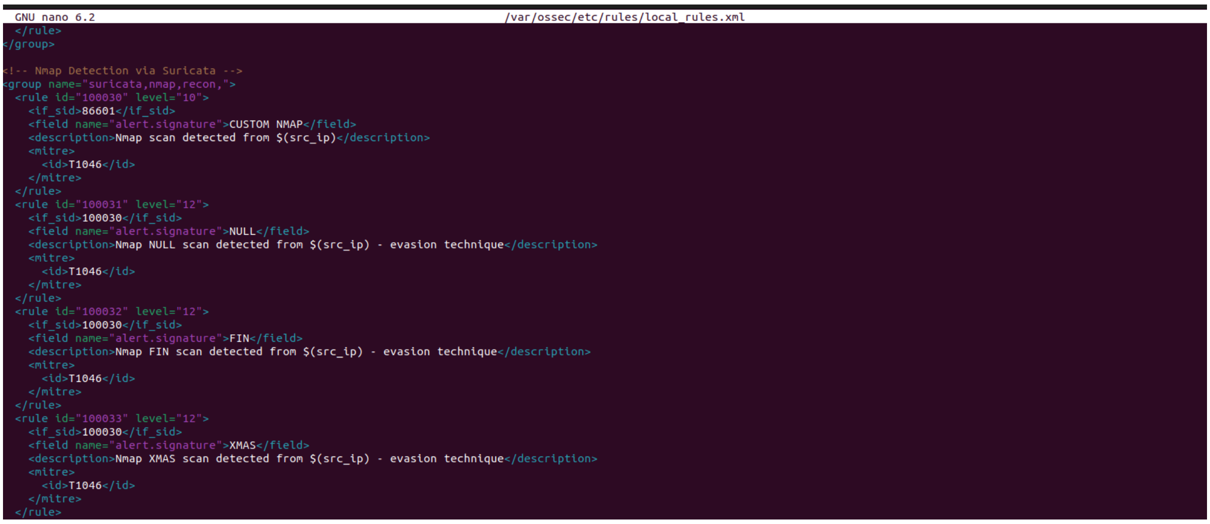
- **Suricata Custom Rule — Nmap Detection**
  A custom rule was added to /var/lib/suricata/rules/custom.rules to detect Nmap scanning patterns at the network level:

```
ubuntu@ubuntu-virtual-machine:~$ cat /var/lib/suricata/rules/custom.rules
# Custom Nmap Detection Rules
alert tcp any any -> $HOME_NET any (msg:"CUSTOM NMAP Scan Detected"; flags:S; threshold: type threshold, track by_src, count 20, seconds 3; classtype:attempted-recon; sid:9000001;
ev:1;)
alert tcp any any -> $HOME_NET any (msg:"CUSTOM NMAP NULL Scan Detected"; flags:0; classtype:attempted-recon; sid:9000002; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"CUSTOM NMAP FIN Scan Detected"; flags:F; classtype:attempted-recon; sid:9000003; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"CUSTOM NMAP XMAS Scan Detected"; flags:FPU; classtype:attempted-recon; sid:9000004; rev:1;)
ubuntu@ubuntu-virtual-machine:~$
```

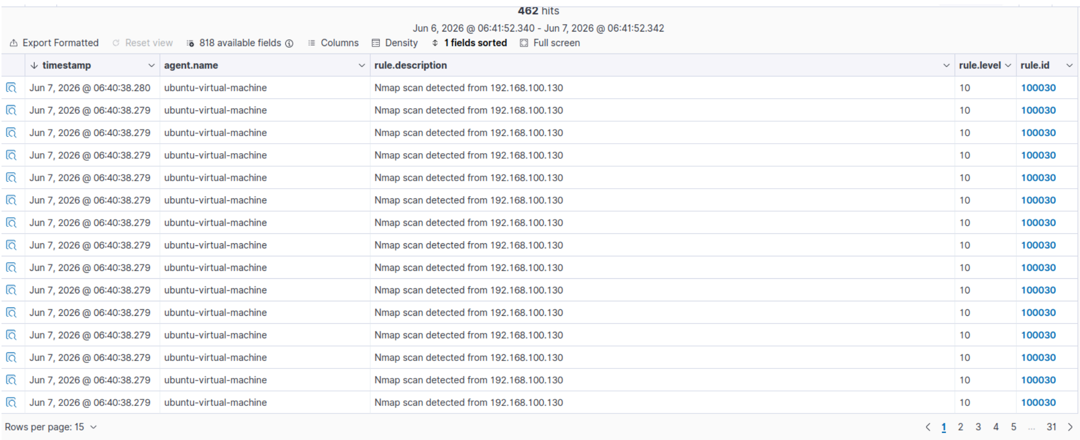
```
7/6/2026 --- 06:10:43 - <info> - cleaning up signature grouping structure... complete
ubuntu@ubuntu-virtual-machine:~$ sudo systemctl restart suricata
ubuntu@ubuntu-virtual-machine:~$ sudo grep "custom.rules" /var/log/suricata/suricata.log
ubuntu@ubuntu-virtual-machine:~$ sudo tail -f /var/log/suricata/eve.json | grep "CUSTOM NMAP"

{"timestamp":"2026-06-07T06:17:45.038489-0400","flow_id":114250111789218 5,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.
128","dest_port":1053,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.038489-0400"}}
{"timestamp":"2026-06-07T06:17:45.043020-0400","flow_id":554998983927820,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":1048,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak"
,"severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.043020-0400"}}
{"timestamp":"2026-06-07T06:17:45.041894-0400","flow_id":129400823674563 8,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.
128","dest_port":5963,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.041894-0400"}}
{"timestamp":"2026-06-07T06:17:45.047088-0400","flow_id":129995247148849 6,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.
128","dest_port":8193,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.047088-0400"}}
{"timestamp":"2026-06-07T06:17:45.047291-0400","flow_id":754360628394171,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":6009,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak"
,"severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.047291-0400"}}
{"timestamp":"2026-06-07T06:17:45.055829-0400","flow_id":339612816497173,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":1068,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak"
,"severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.055829-0400"}}
{"timestamp":"2026-06-07T06:17:45.057157-0400","flow_id":125304713365894 9,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_lp":"192.168.100.
128","dest_port":2042,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.057157-0400"}}
{"timestamp":"2026-06-07T06:17:45.057156-0400","flow_id":395136006217540,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":497,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak",
"severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.057156-0400"}}
{"timestamp":"2026-06-07T06:17:45.058398-0400","flow_id":102299365290089 4,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.
128","dest_port":6839,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.058398-0400"}}
{"timestamp":"2026-06-07T06:17:45.069263-0400","flow_id":782668757798543,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":7496,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak"
,"severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.069263-0400"}}
{"timestamp":"2026-06-07T06:17:45.075617-0400","flow_id":469376663365473,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":2040,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak
","severity":2},"flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":60,"bytes_toclient":0,"start":"2026-06-07T06:17:45.075617-0400"}}
{"timestamp":"2026-06-07T06:17:45.078966-0400","flow_id":819547494495350,"in_iface":"ens33","event_type":"alert","src_ip":"192.168.100.130","src_port":35036,"dest_ip":"192.168.100.1
28","dest_port":631,"proto":"TCP","alert":{"action":"allowed","gid":1,"signature_id":9000001,"rev":1,"signature":"CUSTOM NMAP Scan Detected","category":"Attempted Information Leak"
```

- **Wazuh Custom Rule — Nmap Detection**

A custom rule was added to /var/ossec/etc/rules/local_rules.xml to consume Suricata alerts and generate a higher-level SIEM alert:



```
GNU nano 6.2                                   /var/ossec/etc/rules/local_rules.xml
  </rule>
</group>

<!-- Nmap Detection via Suricata -->
<group name="suricata,nmap,recon,">
  <rule id="100030" level="10">
    <if_sid>86601</if_sid>
    <field name="alert.signature">CUSTOM NMAP</field>
    <description>Nmap scan detected from $(src_ip)</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
  <rule id="100031" level="12">
    <if_sid>100030</if_sid>
    <field name="alert.signature">NULL</field>
    <description>Nmap NULL scan detected from $(src_ip) - evasion technique</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
  <rule id="100032" level="12">
    <if_sid>100030</if_sid>
    <field name="alert.signature">FIN</field>
    <description>Nmap FIN scan detected from $(src_ip) - evasion technique</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
  <rule id="100033" level="12">
    <if_sid>100030</if_sid>
    <field name="alert.signature">XMAS</field>
    <description>Nmap XMAS scan detected from $(src_ip) - evasion technique</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
```



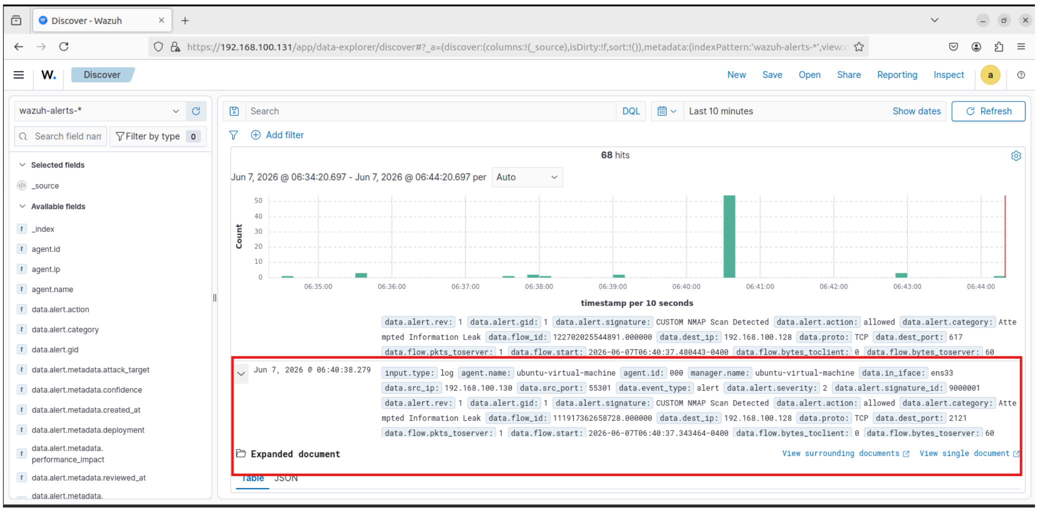
462 hits

Jun 6, 2026 @ 06:41:52.340 - Jun 7, 2026 @ 06:41:52.342

Export Formatted · Reset view · 818 available fields · Columns · Density · 1 fields sorted · Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 7, 2026 @ 06:40:38.280	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030
Jun 7, 2026 @ 06:40:38.279	ubuntu-virtual-machine	Nmap scan detected from 192.168.100.130	10	100030

Rows per page: 15 · 1 2 3 4 5 … 31



Detection Chain

Nmap scan → Suricata detects (sid:9000001) → eve.json → Wazuh rule 100030 fires → Alert in dashboard

SOC Interpretation

If I were sitting in a SOC and saw this alert, my thought process would be:

- The source IP `192.168.100.130` is performing a systematic port scan against the internal Windows endpoint `192.168.100.128`
- No exploitation attempt is observed yet — this is purely reconnaissance activity
- Source IP is internal → could be Kali used for testing OR a compromised internal host conducting lateral reconnaissance
- MITRE ATT&CK technique: T1046 — Network Service Discovery

Recommended action:

- Immediately flag and monitor the source IP for any follow-up activity
- Correlate the scan results with subsequent alerts to detect if discovered open ports were targeted
- Review firewall rules to limit unnecessary port exposure between internal machines

4.2 Attack Scenario —Brute-Force Attack (RDP)

Purpose

The purpose of this scenario was to simulate an attacker attempting to gain unauthorized access to the Windows 10 endpoint by repeatedly trying username and password combinations. Brute-force attacks are one of the most common credential-based attack techniques seen in real SOC environments, and detecting them early is critical to preventing unauthorized access.

Setup

Field	Detail
Target Machine	Windows 10 Workstation

	(192.168.100.128)
Attacker Machine	Kali Linux (192.168.100.130)
Monitoring	Suricata IDS on Ubuntu Server + Wazuh Manager
Tool Used	Hydra
Service Attacked	RDP

Rule Triggered (Custom Rule)

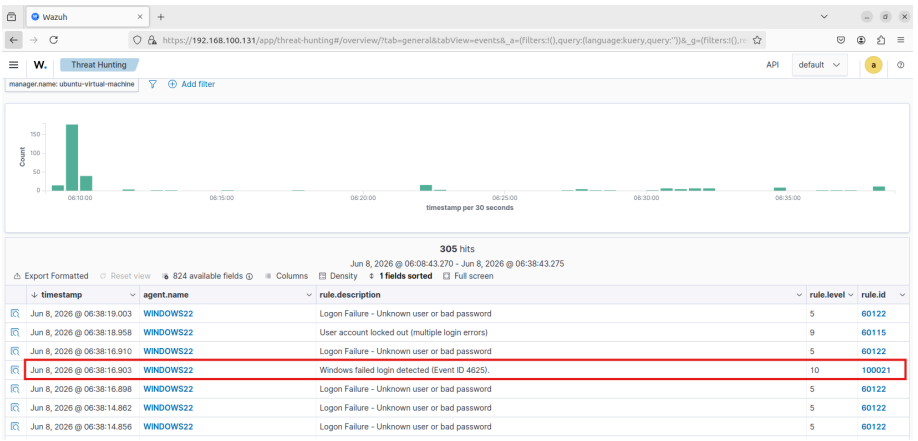
- **Wazuh Custom Rules**

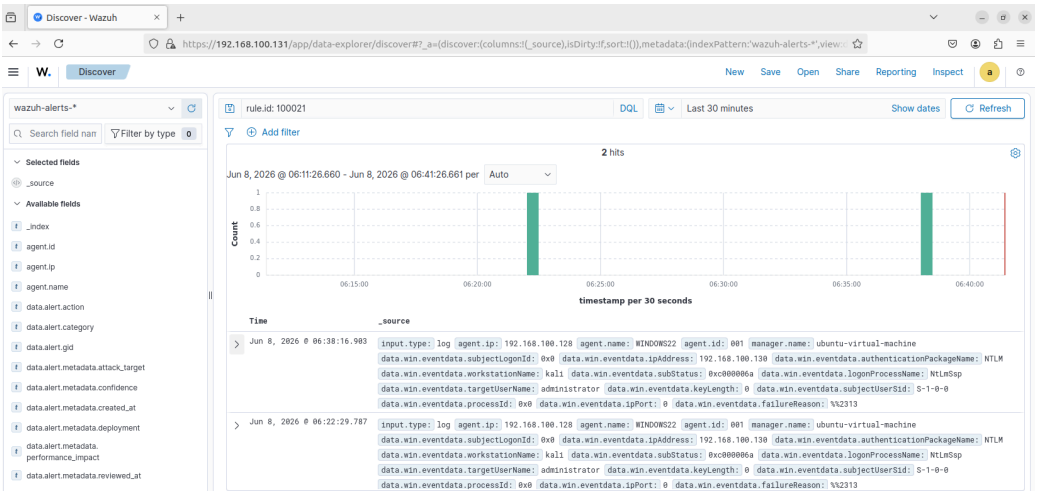
Two custom rules were added to /var/ossec/etc/rules/local_rules.xml:

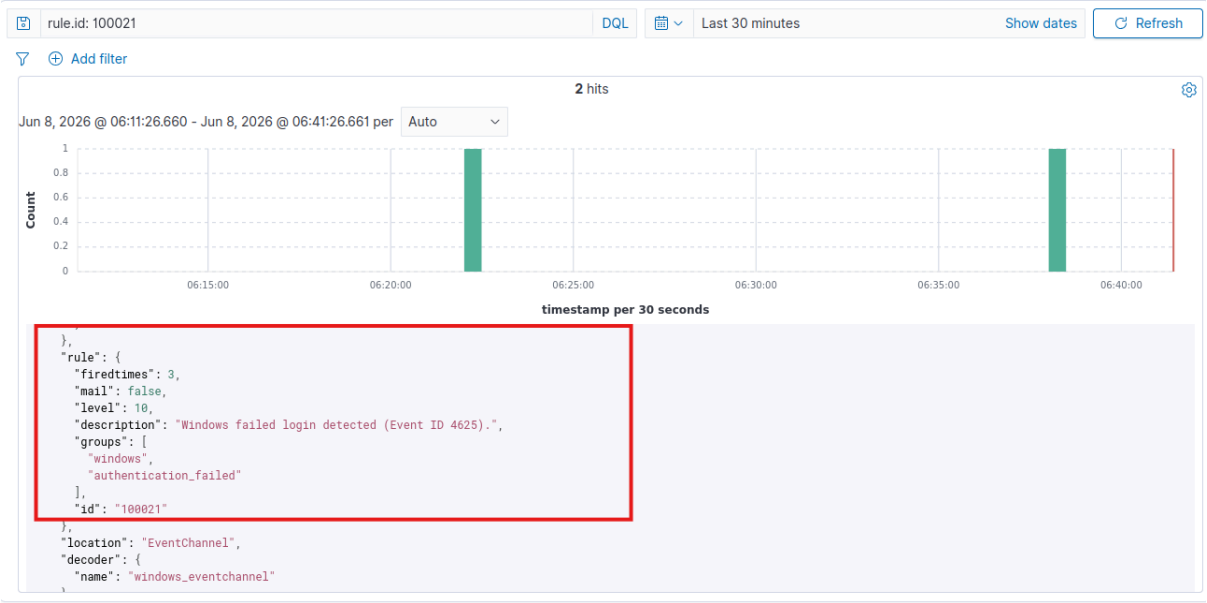
- Rule 100021 — detects each individual failed login via Event ID 4625
- Rule 100025 — detects brute-force pattern when 5 or more failures occur within 60 seconds from the same IP
- Rule 100022 — detects successful RDP login via Event ID 4624 with LogonType 10

```
<!-- Windows Brute Force & RDP -->
<group name="windows,">
  <rule id="100021" level="10">
    <if_sid>60204</if_sid>
    <field name="win.system.eventID">^4625$</field>
    <description>Windows failed login detected (Event ID 4625).</description>
    <group>authentication_failed,</group>
  </rule>

  <rule id="100022" level="10">
    <if_sid>60204</if_sid>
    <field name="win.system.eventID">^4624$</field>
    <field name="win.eventdata.logonType">^10$</field>
    <description>RDP successful login detected.</description>
    <group>rdp,authentication_success,</group>
  </rule>

  <rule id="100023" level="14">
    <if_sid>92200</if_sid>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)(mimikatz|meterpreter)</field>
    <description>Critical: Mimikatz or Meterpreter detected!</description>
    <group>malware,credential_dumping,</group>
  </rule>

  <rule id="100024" level="13">
    <if_sid>92200</if_sid>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)powershell.*-enc</field>
    <description>Suspicious PowerShell encoded command detected.</description>
    <group>malware,powershell,</group>
  </rule>

  <rule id="100025" level="14" frequency="5" timeframe="60">
    <if_matched_sid>100021</if_matched_sid>
    <same_source_ip />
    <description>RDP Brute Force Attack Detected - Multiple Failed Logons from same IP</description>
    <mitre>
      <id>T1110.001</id>
```

```
┌──(kali㉿kali)-[~]
└─$ hydra -l administrator -P /usr/share/wordlists/rockyou.txt rdp://192.168.100.128 -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purp
his is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-08 06:38:07
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking rdp://192.168.100.128:3389/
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "123456" - 1 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "12345" - 2 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "123456789" - 3 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "password" - 4 of 14344399 [child 3] (0/0)
[ERROR] freerdp: The connection failed to establish.
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "iloveyou" - 5 of 14344400 [child 1] (0/1)
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[RE-ATTEMPT] target 192.168.100.128 - login "administrator" - pass "123456" - 5 of 14344400 [child 0] (0/1)
[RE-ATTEMPT] target 192.168.100.128 - login "administrator" - pass "123456789" - 5 of 14344400 [child 2] (0/1)
[ATTEMPT] target 192.168.100.128 - login "administrator" - pass "princess" - 6 of 14344400 [child 0] (0/1)
[ERROR] freerdp: The connection failed to establish.
[RE-ATTEMPT] target 192.168.100.128 - login "administrator" - pass "123456789" - 6 of 14344400 [child 2] (0/1)
```







SOC Interpretation

If I were sitting in a SOC and saw this alert, my thought process would be:

- The attacker IP 192.168.100.130 is repeatedly attempting RDP authentication against the administrator account on 192.168.100.128
- logon type is 3 (Network) — consistent with automated remote login attempts
- Rule 100021 fired multiple times from the same source IP before rule 100025 escalated it to a confirmed brute-force alert
- No successful login observed yet — however rule 100022 must be monitored closely for any follow-up success
- Source IP is internal → could be Kali used for testing OR a compromised internal host attempting lateral movement via RDP
- MITRE ATT&CK technique: T1110.001 — Brute Force: Password Guessing

Recommended action:

- Immediately block or isolate source IP 192.168.100.130
- Review RDP access policy — disable RDP if not required or restrict to specific IPs only
- Enforce account lockout policy after 3-5 failed attempts

4.3 Attack Scenario —SQL Injection

Purpose

The purpose of this scenario was to simulate an attacker performing common web application attacks against the DVWA instance running on the Windows 10 endpoint, and verify whether Wazuh could detect malicious HTTP requests through Apache access log analysis. Web application attacks such as SQL injection, command injection, and cross-site scripting are among the most prevalent attack techniques targeting internet-facing and internal web services. Detecting these attacks at the SIEM level through log analysis is a critical capability for any SOC environment.

Setup

Field	Detail
Target Machine	Windows 10 Workstation (192.168.100.128)
Attacker Machine	Kali Linux (192.168.100.130)
Monitoring	Wazuh Manager + Apache Access Log Forwarding
Tool Used	curl
Service Attacked	DVWA (Damn Vulnerable Web Application) via HTTP port 80

Rules Triggered (Custom Rules)

custom rule were added to /var/ossec/etc/rules/local_rules.xml:

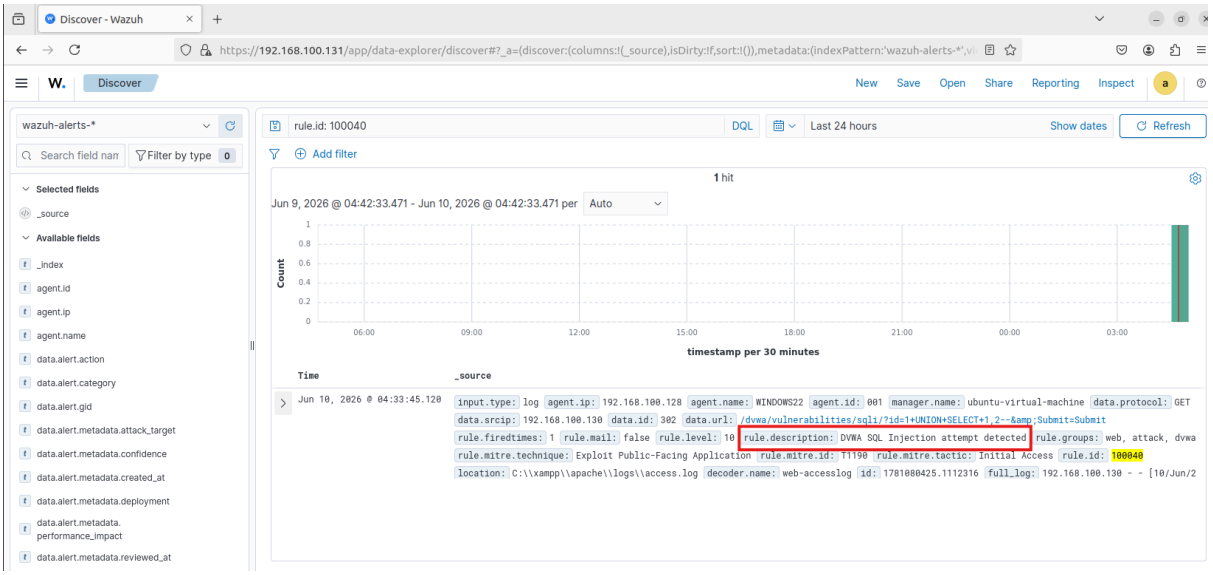
- Rule 100040 — detects SQL injection patterns in the URL such as UNION SELECT, OR 1=1, and ORDER BY

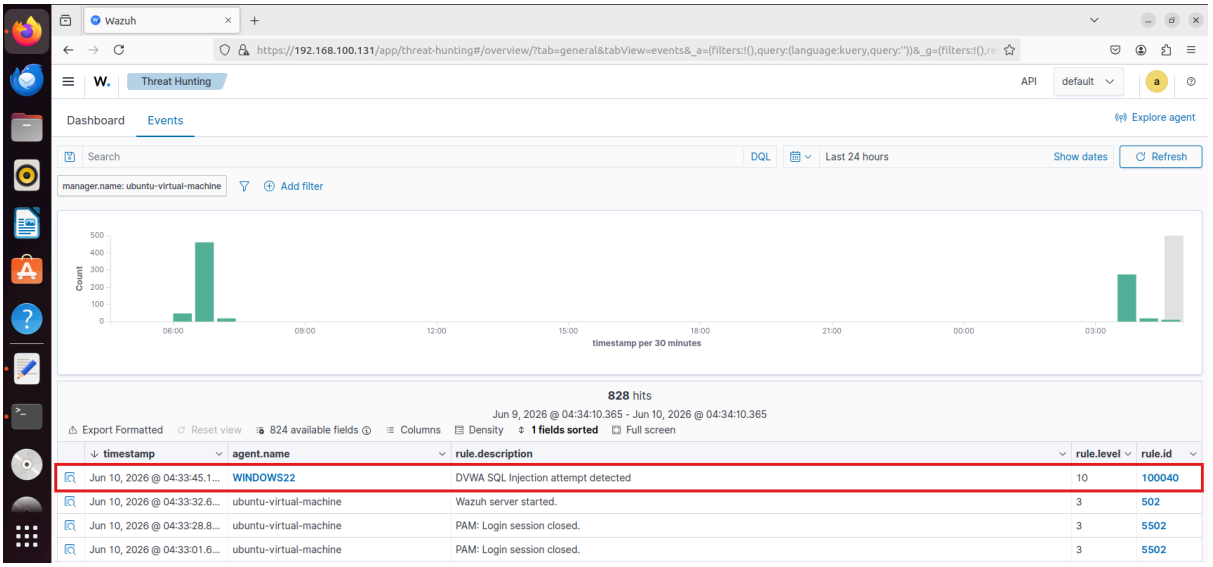
```
 GNU nano 6.2                                  /var/ossec/etc/rules/local_rules.xml
    <if_sid>100030</if_sid>
    <field name="alert.signature">XMAS</field>
    <description>Nmap XMAS scan detected from $(src_ip) - evasion technique</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
</group>

<!-- DVWA Web Attack Detection -->
<group name="web,attack,dvwa,">
  <rule id="100040" level="10">
    <if_sid>31100,31101</if_sid>
    <url>union+select|union%20select|1=1|'or'|order+by|group+by</url>
    <description>DVWA SQL Injection attempt detected</description>
    <mitre>
      <id>T1190</id>
    </mitre>
  </rule>
  <rule id="100041" level="12">
    <if_sid>31100,31101</if_sid>
    <url>%3Bwhoami|%3Bcat|%3Bls|%7Cwhoami|%26whoami|/etc/passwd|cmd=|exec=</url>
    <description>DVWA Command Injection attempt detected</description>
    <mitre>
      <id>T1059</id>
    </mitre>
  </rule>
  <rule id="100042" level="10">
    <if_sid>31100,31101</if_sid>
    <url>%3Cscript%3E|onerror=|onload=|alert%281%29|javascript:</url>
    <description>DVWA XSS attempt detected</description>
    <mitre>
      <id>T1059.007</id>
    </mitre>
  </rule>
</group>
```

- **Kali Command**

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.100.128/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+1,2--&Submit=Submit"

┌──(kali㉿kali)-[~]
└─$
```

- **Wazuh Events**





SOC Interpretation

If I were sitting in a SOC and saw this alert, my thought process would be:

- Source IP 192.168.100.130 is sending a crafted HTTP GET request containing a SQL injection payload against the web application on 192.168.100.128
- Rule 100040 fired on a URL containing UNION+SELECT ,the attacker is attempting to append additional SQL statements to extract data from the backend database
- HTTP 302 redirect was returned, indicating the request reached the application layer and was processed

- Source IP is internal → could be Kali used for testing OR a compromised internal host attempting to extract sensitive data from an internal web application

Recommended action:

- Immediately investigate all requests from source IP 192.168.100.130 for further malicious activity
- Enforce parameterized queries and input validation on the web application to prevent SQL injection
- Restrict web application access to authorized users and IP ranges only
- Correlate this alert with database logs to determine if any data was successfully extracted

4.4 Attack Scenario —Cross-Site Scripting via DVWA

Purpose

The purpose of this scenario was to simulate an attacker performing a reflected Cross-Site Scripting attack against the DVWA instance running on the Windows 10 endpoint, and verify whether Wazuh could detect the malicious JavaScript payload through Apache access log analysis. XSS attacks are one of the most common web application vulnerabilities and are frequently used to steal session cookies, redirect users to malicious pages, or execute unauthorized actions in the victim's browser.

Setup

Field	Detail
Target Machine	Windows 10 Workstation (192.168.100.128)
Attacker Machine	Kali Linux (192.168.100.130)
Monitoring	Wazuh Manager + Apache Access Log Forwarding
Tool Used	curl
Service Attacked	DVWA XSS (Reflected) module via HTTP port 80

Rules Triggered (Custom Rules)

custom rule were added to /var/ossec/etc/rules/local_rules.xml:

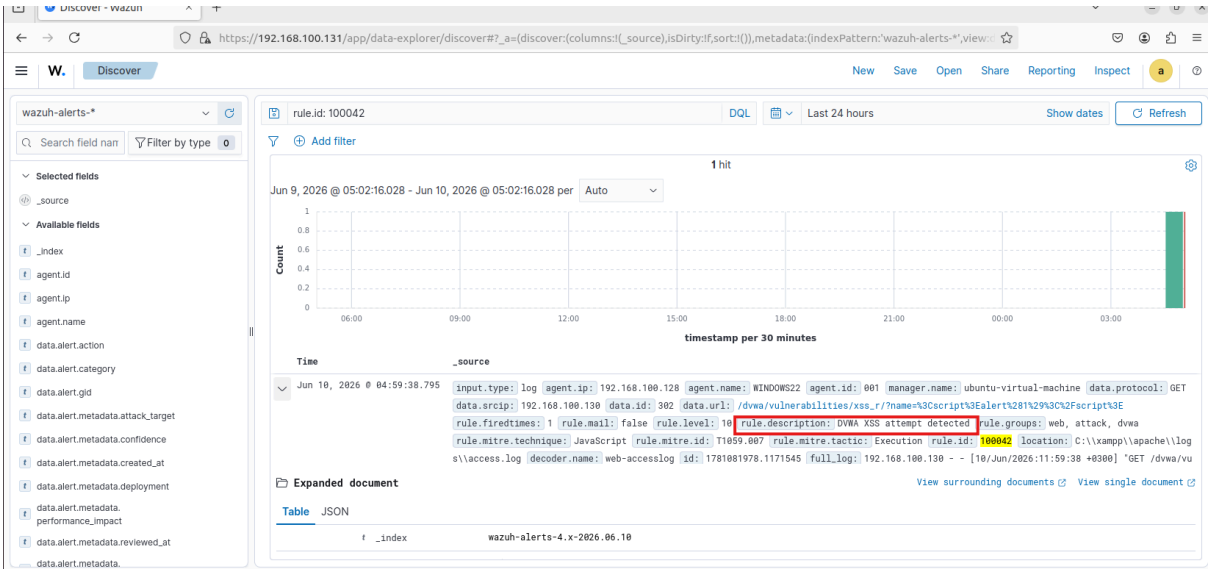
- Rule 100042 was added to detect XSS patterns in HTTP request URLs:

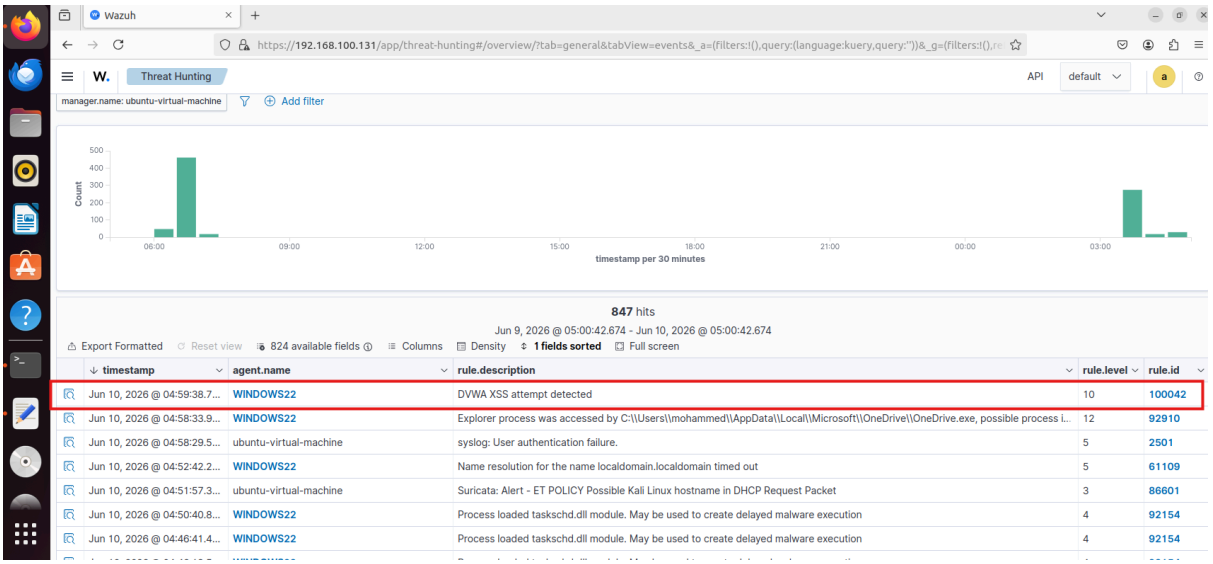
```
 GNU nano 6.2                                                    /var/ossec/etc/rules/local_rules.xml
    <if_sid>100030</if_sid>
    <field name="alert.signature">XMAS</field>
    <description>Nmap XMAS scan detected from $(src_ip) - evasion technique</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
</group>

<!-- DVWA Web Attack Detection -->
<group name="web,attack,dvwa,">
  <rule id="100040" level="10">
    <if_sid>31100,31101</if_sid>
    <url>union+select|union%20select|1=1|'or'|order+by|group+by</url>
    <description>DVWA SQL Injection attempt detected</description>
    <mitre>
      <id>T1190</id>
    </mitre>
  </rule>
  <rule id="100041" level="12">
    <if_sid>31100,31101</if_sid>
    <url>%3Bwhoami|%3Bcat|%3Bls|%7Cwhoami|%26whoami|/etc/passwd|cmd=|exec=</url>
    <description>DVWA Command Injection attempt detected</description>
    <mitre>
      <id>T1059</id>
    </mitre>
  </rule>
  <rule id="100042" level="10">
    <if_sid>31100,31101</if_sid>
    <url>%3Cscript%3E|onerror=|onload=|alert%281%29|javascript:</url>
    <description>DVWA XSS attempt detected</description>
    <mitre>
      <id>T1059.007</id>
    </mitre>
  </rule>
</group>
```

- **Kali Command**

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.100.128/dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%281%29%3C%2Fscript%3E"

┌──(kali㉿kali)-[~]
└─$
```

- **Wazuh Events**





SOC Interpretation

If I were sitting in a SOC and saw this alert, my thought process would be:

- Source IP 192.168.100.130 is sending a crafted HTTP GET request containing a JavaScript payload in the URL parameter against the web application on 192.168.100.128
- Rule 100042 fired on a URL containing `%3Cscript%3E` — the attacker is injecting a script tag attempting to execute JavaScript in the victim's browser
- This is a reflected XSS attempt — the malicious script is embedded in the URL and would execute if a victim clicks the crafted link

- HTTP 302 redirect was returned, indicating the request reached the application layer
- Source IP is internal → could be Kali used for testing OR a compromised internal host attempting to steal session cookies or redirect users to malicious pages
- MITRE ATT&CK technique: T1059.007 — Command and Scripting Interpreter: JavaScript

Recommended action:

- Investigate all requests from source IP 192.168.100.130 for further malicious activity
- Enforce output encoding and Content Security Policy (CSP) headers on the web application
- Sanitize all user-supplied input before rendering it in the browser
- Monitor for any session hijacking attempts following this alert

5. Conclusion

This project gave me practical, hands-on experience building and operating a real SOC monitoring environment from scratch. Rather than just reading about SIEM concepts, I was able to deploy the tools, generate real attacks, and observe how security events are detected and analyzed end to end.

Throughout the project I learned several important lessons. First, visibility is everything in a SOC , without proper log collection from endpoints, network traffic, and web applications, attacks simply go undetected. Adding Apache log forwarding for DVWA showed me how easy it is to miss an entire attack surface if log sources are not properly configured.

Second, detection rules must be carefully written and tested. During this project I wrote custom rules for Wazuh and Suricata and learned that poorly written rules either miss attacks or fire on the wrong thing. Getting the rules right required understanding both the attack technique and how the log data looks at the SIEM level.

Third, every alert tells a story. Learning to read an alert, map it to a MITRE ATT&CK technique, and think about what the attacker was trying to do is the core skill of a SOC analyst. This project helped me practice that thinking across multiple attack types including reconnaissance, brute force, and web application attacks.

If I were to expand this lab in the future, I would add more endpoints, simulate more advanced attack chains, and integrate threat intelligence feeds to enrich alerts with external context.

Overall this project demonstrated that building a homelab is one of the most effective ways to develop real defensive security skills, and I plan to continue expanding it as I grow in my cybersecurity career.